HDOC-68

Recursive Decimal to Binary Conversion

Problem Statement

Write a recursive function in C to convert a decimal number into its binary representation.

Algorithm / Logic

1. Read a non-negative decimal integer from the user.

2. If the number is 0, print 0 directly.

3. Otherwise, call the recursive function decimalToBinary(n).

4. In the recursive function, if n is greater than 1, recursively call decimalToBinary(n / 2).

5. After the recursive call returns, print n % 2. This prints the binary digits from most significant to least significant.

6. Stop when all recursive calls are completed.

C Program

```
#include <stdio.h>

void decimalToBinary(unsigned int n)
{
   if (n > 1)
      decimalToBinary(n / 2);

   printf("%u", n % 2);
}

int main()
{
   unsigned int decimal;

   printf("Enter a decimal number: ");
   scanf("%u", &decimal);

   printf("Binary representation: ");

   if (decimal == 0)
      printf("0");
   else
      decimalToBinary(decimal);

   printf("\n");
   return 0;
}
```

Explanation

The function decimalToBinary() uses recursion. For a decimal number n, the binary representation is obtained by repeatedly dividing n by 2. The remainder of each division is either 0 or 1. Because the remainders are generated from least significant to most significant, the function first makes the recursive call with n / 2 and prints n % 2 while the calls return. This automatically displays the bits in the correct order.

Example: for 10, the recursive calls use 10 -> 5 -> 2 -> 1. While returning, the remainders are printed as 1, 0, 1, 0, giving 1010.

Test Cases

Decimal Input	Expected Binary Output	Result
0	0	Pass
1	1	Pass
5	101	Pass
10	1010	Pass
25	11001	Pass
100	1100100	Pass

Sample Output

Enter a decimal number: 25
Binary representation: 11001

Result

The program successfully converts a non-negative decimal number into its binary representation using a recursive function.